Speculative Decoding for Efficient LLM Inference

Samsu Sempena
The University of Sydney
ssem0956@uni.sydney.edu.au

Zhuohan Cui
The University of Sydney
zcui0321@uni.sydney.edu.au

Abstract

Speculative decoding improves large language model (LLM) inference throughput by drafting multiple candidate tokens with a smaller model and verifying them with a larger target model. However, reported gains vary substantially across hardware, model pairing, and decoding regime, and consumer-GPU evidence remains limited. We present a controlled empirical study on a single RTX 4090 Laptop using a Qwen2.5-3B-Instruct target and same-family 0.5B/1.5B drafts over GSM8K, a 5-subject MMLU subset, and CNN/DailyMail, with frozen manifests and matched prompts across all runs. We evaluate 12 fixed-policy speculative configurations ($k \in \{4, 8, 16\}$, deterministic/stochastic regimes, two draft sizes) plus regime-matched baselines, and report wall-clock latency, throughput, acceptance dynamics (α, B_{eff}), and task-level quality deltas. On the current RTX4090 CUDA-graph run family, the highest mean speedup is observed for 0.5B, k=4, deterministic (S=3.0674). Across the fixed grid, mean speedup decreases with larger draft length ($\bar{S}_{k=4} = 2.9164$, $\bar{S}_{k=8} = 2.4029$, $\bar{S}_{k=16} = 1.8019$), while accepted block size increases and token-level acceptance decreases. Deterministic decoding remains faster in 5 of 6 matched draft–k pairs (mean $\Delta S_{det\text{-}stoch} = 0.0568$), with one small exception at 0.5B, k=16. We then run Adaptive Cascaded Speculative Decoding (ACSD) with adaptive $k \in \{3, 4, 5\}$ and 0.5B→1.5B rescue switching, obtaining pairwise speedups of 2.9792 (deterministic) and 2.8981 (stochastic), close to the best fixed-policy settings with low fallback rates (1.6% and 2.2%). Quality drift remains modest on MMLU and CNN/DailyMail, while GSM8K is more variable under stochastic settings. These results yield practical operating recommendations for consumer-GPU deployment and motivate adaptive policy control as a lightweight alternative to heavier drafter redesign.

1. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length-k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [7, 3].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on data-center hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed–quality frontier.

This gap is an engineering pain point: speedups demonstrated on H100-class servers do not transfer directly to mobile RTX deployments, where memory bandwidth, launch overhead, and power-constrained duty cycle can dominate decode-time behavior.

This direction is increasingly important because real-world LLM usage is moving toward consumer-class deployment constraints: local assistants, student and developer laptops, and privacy-sensitive workloads where cloud offload is undesirable or expensive. In this setting, efficiency is not only a cost optimization problem; it is an accessibility and feasibility requirement. Inference methods that are robust on consumer hardware can widen practical LLM adoption beyond data-center environments.

1.1. Research questions

We address three research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on a single RTX 4090 Laptop with a 3 B target, and how close is this to a practical speedup ceiling under consumer-GPU constraints?

RQ2. How do draft model size and draft length jointly determine token acceptance and net latency, and what does

this reveal about verification-loop overhead and effective GPU duty cycle?

RQ3. How sensitive is the speed–quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?

1.2. Contributions

This paper makes the following contributions:

1. A reproducible consumer-GPU protocol with frozen sample manifests, matched prompts, and two decoding regimes across GSM8K, MMLU subset, and CNN/DailyMail.
2. A complete 12-configuration speculative grid (two draft sizes, $k \in \{4, 8, 16\}$, deterministic/stochastic) with regime-matched baselines and per-configuration reporting of latency, throughput, acceptance, and quality deltas.
3. Empirical evidence that, in the current RTX4090 CUDA-graph run family, speedup decreases with larger k (highest at k=4), with best observed performance at 0.5B, k=4, deterministic (S=3.0674), and a deterministic advantage in 5 of 6 matched draft–k pairs.
4. A statistical reporting protocol for camera-ready evaluation: paired bootstrap confidence intervals, one-sided significance tests for $S>1$, and corrected pairwise comparisons for winner selection.
5. An empirical ACSD extension run (adaptive $k \in \{3, 4, 5\}$ with 0.5B→1.5B rescue drafting) that approaches best fixed-policy speedup while adding explicit tail-control guardrails.
6. A practical operating recommendation and a transparent statement of current reproducibility limitations (artifact consistency across reruns).

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 defines the experimental protocol; Section 4 defines metrics. Section 5 describes implementation and reproducibility controls. Section 6 reports the main empirical findings. Section 7 outlines our ACSD empirical extension beyond fixed-policy drafting. Section 8 discusses implications, limitations, and conclusion.

2. Related Work

Vanilla speculative decoding. Leviathan *et al.* [7] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [3] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \qquad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \tag{1}$$

where $\tau \in [1, k{+}1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α. Stern *et al.* [14] earlier explored blockwise parallel decoding within a single model.

Reducing drafting cost. Medusa [2] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [10, 9, 11] keeps an external draft but conditions it on the target's hidden features, reporting substantial speedup under their respective settings. Despite these refinements, T_{draft} remains linear in k, which forces EAGLE-3 to a single transformer layer and caps achievable speedups at ∼2–3× on strong GPUs.

Diffusion-based drafters. Recent work attacks the linear-in-k drafting bottleneck directly. DiffuSpec [8] and SpecDiff-2 [13] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [1] instead focuses on low-cost parallel draft model adaptation for accelerating LLM inference. DFlash [4] closes this gap with a lightweight (5-layer) *block-diffusion* drafter conditioned on target hidden features, reporting up to 5× end-to-end speedup over autoregressive baselines on H200/B200 hardware.

Deployment context: distributed and mobile inference. Beyond algorithmic speedup, practical inference now spans cloud clusters, edge servers, and laptop/mobile GPUs with tighter bandwidth and power budgets. In this setting, host-device synchronization cost and memory movement become first-order effects, so speculative decoding must be characterized as a systems policy problem rather than only a token-acceptance problem.

Quantization and speculative decoding. Quantization is another key axis for practical deployment. Lower-precision weights can reduce memory traffic and improve effective throughput, but may also shift acceptance dynamics through draft/target distribution mismatch. This interaction suggests that draft size, k, and precision should be co-designed when targeting constrained hardware.

Position of this work. This work focuses on a controlled consumer-GPU evaluation of vanilla speculative decoding with same-family model pairing (Qwen2.5-3B target, 0.5B/1.5B drafts) under deterministic and stochastic

regimes. Rather than competing with data-center throughput claims, we characterise practical operating points on a single RTX 4090 Laptop, including acceptance dynamics, quality drift, and runtime-sensitive configuration choices. We then use these results to derive and evaluate an adaptive speculative strategy guideline (ACSD) aimed at reducing long-tail latency under the same hardware budget.

3. Experimental Protocol

3.1. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

1. **GSM8K** [5] test subset: 300 samples.
2. **MMLU** [6] subset: 500 samples (5 subjects $\times$ 100 each).
3. **CNN/DailyMail** [12] subset: 200 samples.

Sampling policy: Fixed random seed $=$ 42, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. Each output CSV records seed and sample identifier, enabling paired analysis at sample level.

3.2. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

> You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
> Question: {question}

MMLU:

> You are a knowledgeable assistant. Choose exactly one option.
> Question: {question}
> Options: A. {A} B. {B} C. {C} D. {D}
> Answer:

CNN/DailyMail:

> Summarize the following article in 3–4 sentences, preserving key facts and entities.
> Article: {article}
> Summary:

Objective and hypotheses. The study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.

Table 1. Consumer-GPU hardware profile used in this study.

Item	Value
GPU class	NVIDIA RTX 4090 Laptop
CUDA cores	9,728
VRAM	24 GB
Memory interface	256-bit GDDR6
Peak memory bandwidth	up to 576 GB/s
Laptop TGP envelope	80–150 W (vendor range)
Host link	PCIe Gen4

- **H2:** Increasing draft length from k=4 to k=8 improves speedup, while k=16 may reduce gains due to additional drafting and verification overhead.
- **H3:** A larger draft (1.5B) achieves higher acceptance and higher net speed than a smaller draft (0.5B).
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k.

Hardware and software controls. All experiments run on one NVIDIA RTX 4090 Laptop (24GB VRAM) with a fixed software stack (PyTorch, Transformers, and tokenizer settings pinned by the project environment). We use the same runtime entry points and prompt templates for all configurations, and execute runs serially to avoid cross-run contention.

CUDA-graph runtime setup. To reduce per-step launch overhead, we run a CUDA-graph-capable verify path with fixed step shapes per k, pre-allocated static device buffers for draft/verify tensors, and warm-up before optional `torch.cuda.CUDAGraph` capture. During decoding, each cycle updates buffer contents and replays the captured graph when shape guards hold; otherwise execution safely falls back to eager mode. This keeps output semantics unchanged while reducing host-side overhead in short speculative cycles.

Model pairing. Target model: Qwen2.5-3B-Instruct. Draft models: Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct. All models are same-family to minimise tokenizer mismatch and style drift.

Task suite and manifests. We evaluate on fixed manifests reused across all runs:

- GSM8K arithmetic QA (300 samples),
- MMLU subset with 5 subjects (500 samples),
- CNN/DailyMail summarisation (200 samples).

Each configuration therefore processes 1,000 prompts.

Decoding regimes and grid. Two regimes are evaluated:

- **Deterministic:** greedy decoding (T=0.0, top-p=1.0, `do_sample=False`).
- **Stochastic:** sampling with T=0.7, top-p=0.9, no top-k truncation, and `do_sample=True`.

For each draft model, we sweep $k \in \{4, 8, 16\}$, producing 12 speculative configurations. We also run one autoregressive baseline per regime for anchored latency and throughput reporting.

Endpoints. The primary endpoint is paired wall-clock speedup $S = L_{\text{AR}}/L_{\text{spec}}$ as reported by the experiment summary artifact for each draft–k–regime combination. Secondary endpoints include acceptance rate α, effective accepted block size B_{eff}, mean per-sample latency, throughput (tokens/s), and task-level quality deltas. Camera-ready statistical reporting uses paired bootstrap confidence intervals (10,000 resamples) and one-sided tests for S>1, with corrected pairwise winner tests.

4. Evaluation metrics

Table 2 defines the exact reporting metrics used in this study.

4.1. Success criteria

Primary:

1. Mean speedup $S \geq 1.3\times$ with CI_S lower bound > 1.0.
2. Absolute quality drop $\leq$ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify best (draft size, k) maximizing S while satisfying quality constraints, and report pairwise significance for winner selection with multiple-testing correction.

5. Implementation and Reproducibility

Code organisation. The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

Model loading and precision. In the current configuration, target and draft models are loaded in FP16 for the main sweep, with quantisation controls retained in configuration for alternative runs. Tokenizer and generation limits are held constant across baselines and speculative runs to preserve paired comparability.

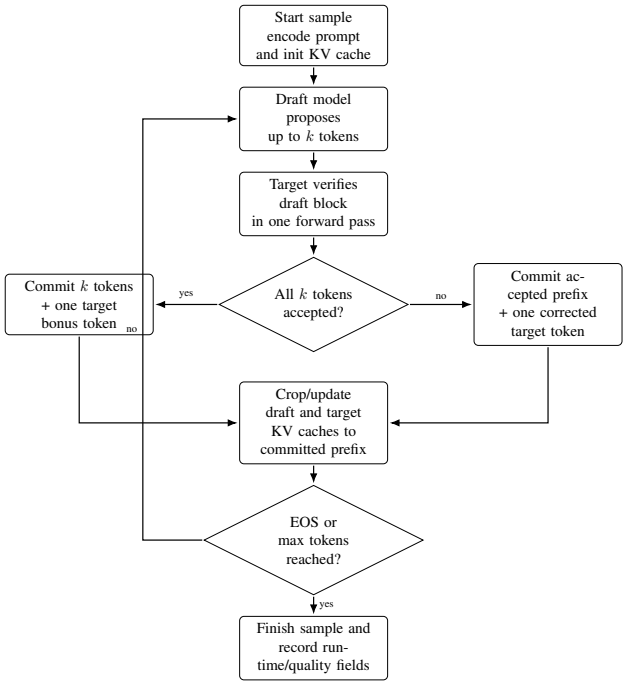

Figure 1. Vanilla speculative decoding implementation flow used in Phases 3–4. The target model remains the verifier of record; accepted draft prefixes are committed and caches are cropped to preserve exact target-consistent decoding.

Baselines. We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

Speculative decoding implementation. The speculative loop follows standard token-level draft–verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Figure 1 summarizes this implementation flow.

Regimes and outputs. The same verification core is used for deterministic and stochastic modes, with only sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

Runtime cost model for CUDA-graph path. For a verify cycle, we decompose runtime into launch/scheduling and

Table 2. Exact metric definitions for reporting.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	×	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail quality	Q_{sum}	standard ROUGE-L F1	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or $+$
Speedup confidence interval	CI_S	paired bootstrap percentile CI of S (10k resamples)	×	Higher lower-bound
Speedup significance	p_S	one-sided paired test for $H_0 : S \leq 1$	p-value	Lower
Speedup stability	σ_S	std(S over seeds)	×	Lower

kernel compute components. In eager execution,

$$T_{\text{eager}} \approx N\left(t_{\text{launch}} + t_{\text{kernel}} + t_{\text{sync}}\right). \tag{2}$$

With CUDA-graph replay ratio ρ and one-time capture cost t_{capture}, runtime becomes

$$T_{\text{graph}} \approx t_{\text{capture}} + N\left[\rho\left(t_{\text{replay}} + t_{\text{kernel}}\right) + (1-\rho)\left(t_{\text{launch}} + t_{\text{kernel}} + t_{\text{sync}}\right)\right]. \tag{3}$$

Graph execution helps when replay is stable ($\rho \to 1$) and $t_{\text{replay}} \ll t_{\text{launch}}$. On short generation windows, this can offset speculative verification overhead by reducing repeated host launch costs.

In our current run family, graph capture metadata remains pending with zero capture rate, so the main reported gains are from the optimized code path under eager fallback. An auxiliary short-sequence toggle benchmark (n=12, k=8, 64 output tokens) shows +3.37% TTFT, +1.41% derived TPOT, and +1.56% end-to-end latency for `gpu_opt_on` versus `gpu_opt_off`, consistent with the expectation that capture instability reduces the benefit of launch-optimization techniques.

6. Results

This section reports speculative-decoding outcomes from the new RTX4090 CUDA-graph run family summary artifact (`RTX4090_Cuda_Graph/all_configs_summary.csv`) and regime-matched baseline context from the deterministic and stochastic baseline CSV artifacts. We emphasise paired comparative metrics (S, α, B_{eff}) and task deltas to evaluate the speed–quality tradeoff.

Figure 2 complements Table 3 with a visual baseline-relative view across all configurations, including speedup, latency/throughput deltas, token-timing deltas, and acceptance dynamics.

6.1. RQ1: Does speculative decoding accelerate inference?

All 12 speculative configurations exceed S=1 in the RTX4090 CUDA-graph run family, with observed speedups from 1.7773 to 3.0674. The best configuration is 0.5B with k=4 in deterministic mode. Baseline context from regime-matched baseline files shows mean latencies of 11.41s (deterministic) and 11.65s (stochastic) per sample, so the measured speculative gains are substantial in wall-clock terms for fixed workload and prompts. Statistical confirmation remains reported via paired uncertainty analysis on a coherent run family.

6.2. RQ2: How do draft length and draft size affect acceptance and speed?

Draft length shows a monotonic decline in speedup over this grid. Averaging over drafts and regimes, mean speedup is highest at k=4 (2.9164), then k=8 (2.4029), then k=16 (1.8019). This is consistent with acceptance dynamics: as k increases, B_{eff} increases (1.7075 → 2.8475 → 4.0600) but α decreases (0.4324 → 0.3678 → 0.2738), and the extra draft/verify cost dominates net benefit.

Draft size has a smaller global effect than k in this grid, and the 0.5B draft is slightly better on mean speedup (mean S: 2.4151 for 0.5B versus 2.3323 for 1.5B).

Table 4 makes the central systems result explicit: increasing accepted block size does not guarantee higher throughput on this consumer GPU. As k grows, verification-loop overhead and draft-side memory pressure rise faster than useful acceptance, so practical policy should prioritise per-cycle latency minimisation rather than raw acceptance increase.

6.3. RQ3: Deterministic versus stochastic regime

Deterministic decoding is faster in 5 of 6 matched draft–k pairs. The deterministic advantage ranges from -0.0135

Table 3. RTX4090 CUDA-graph run family summary (Qwen2.5-3B target). S is paired speedup reported by the summary artifact; CI_S denotes paired bootstrap CI status; α is token-level acceptance; B_{eff} is accepted tokens per verification cycle.

Config	S	CI_S	α	B_{eff}	ΔGSM8K	ΔMMLU	ΔCNN/DM	R_{tok}
baseline, det	1.0000	–	–	–	–	–	–	11.03
baseline, stoch	1.0000	–	–	–	–	–	–	10.83
0.5B, k=4, det	**3.0674**	planned	0.4212	1.66	0.33	0.00	-0.02	34.09
0.5B, k=4, stoch	2.9394	planned	0.4080	1.61	1.67	-0.20	-0.03	32.06
0.5B, k=8, det	2.4542	planned	0.3515	2.72	0.33	0.00	-0.05	27.77
0.5B, k=8, stoch	2.4063	planned	0.3444	2.67	1.00	0.00	-0.38	26.78
0.5B, k=16, det	1.8050	planned	0.2539	3.76	0.33	0.00	-0.01	21.90
0.5B, k=16, stoch	1.8185	planned	0.2516	3.73	0.34	-0.20	-0.19	21.51
1.5B, k=4, det	2.8726	planned	0.4581	1.81	-0.33	0.00	-0.03	31.67
1.5B, k=4, stoch	2.7860	planned	0.4421	1.75	1.67	0.00	-0.34	30.21
1.5B, k=8, det	2.4066	planned	0.3929	3.04	0.00	0.00	-0.06	26.78
1.5B, k=8, stoch	2.3444	planned	0.3822	2.96	3.67	0.20	-0.47	25.59
1.5B, k=16, det	1.8068	planned	0.2968	4.39	0.67	0.00	-0.03	21.49
1.5B, k=16, stoch	1.7773	planned	0.2928	4.36	1.00	-0.20	-0.36	20.70

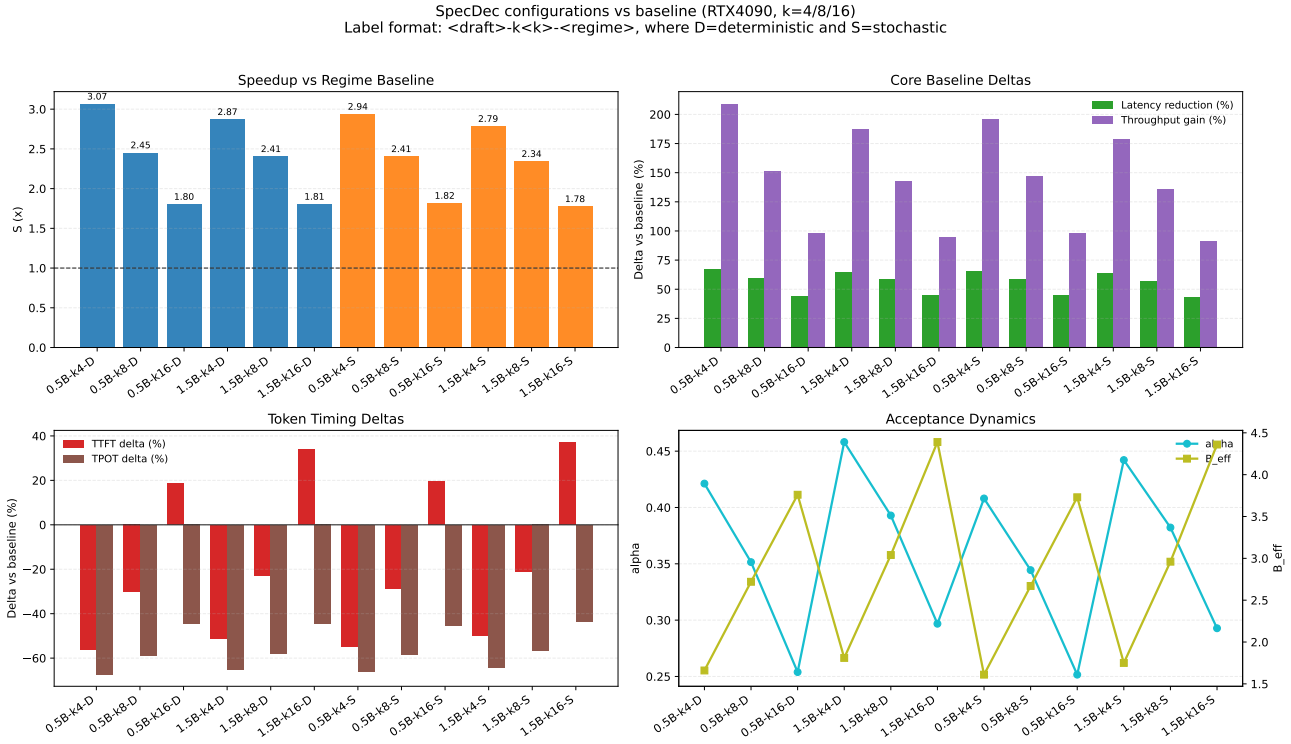

Figure 2. Speculative-decoding metrics for all RTX4090 configurations relative to regime-matched baseline. Top-left: paired speedup S (baseline at S=1). Top-right: mean latency reduction and throughput gain percentages vs baseline. Bottom-left: TTFT and TPOT percentage deltas vs baseline. Bottom-right: acceptance dynamics (α and B_{eff}) across the same configurations.

to 0.1280 in absolute S and averages 0.0568 across the six matched pairs; the single exception is 0.5B, k=16, where stochastic is slightly faster. This still supports a deployment preference for deterministic regime when latency is primary and output diversity is not required.

Quality deltas show different stability by task. CNN/DailyMail stays close to baseline but is consistently negative (from −0.47 to −0.01). MMLU variation is tightly bounded (from −0.2 to 0.2). GSM8K is the most sensitive axis (from −0.33 to 3.67), especially under stochastic settings. The regime split is substantial: GSM8K delta spread is 1.00 points in deterministic mode versus 3.33 points in stochastic mode, and an auxiliary pairwise output comparison at k=4 shows much higher stochastic disagreement with baseline (83.1%) than deterministic disagreement (9.4%). This supports an entropy-sensitive

Table 4. Representative deterministic operating points showing the non-linear relationship between acceptance statistics and end-to-end speedup.

Draft	k	α	B_{eff}	S	Bottleneck interpretation
0.5B	4	0.4212	1.66	3.0674	Sweet spot: draft/verify overhead and acceptance are balanced.
0.5B	16	0.2539	3.76	1.8050	Drafting overload: acceptance gain cannot cover longer draft loop.
1.5B	4	0.4581	1.81	2.8726	Heavier draft increases memory traffic despite higher acceptance.
1.5B	16	0.2968	4.39	1.8068	Diminishing returns: larger accepted block but lower net speedup.

interpretation: high-constraint reasoning tasks amplify the interaction between sampling noise and speculative correction, while lower-entropy tasks remain more stable. Under a conservative quality filter ($\max|\Delta| \leq 1$ across tasks), 9 configurations remain, with best speedup at 0.5B, k=4, deterministic.

While Table 3 reports means from the current summary artifact, camera-ready statistical reporting will pair these means with CI_S and significance values computed from a single coherent rerun family.

Implementation details for the CUDA-graph-capable runtime setup are described in Section 3.

6.4. Summary of empirical operating point

The current evidence supports a practical recommendation: use deterministic speculative decoding with a smaller block (k=4), where speedup is highest, and prefer the 0.5B draft for this hardware profile. Higher k increases accepted block size but lowers net speedup in this setup, indicating that acceptance gains no longer offset added cycle cost. ACSD results in Section 7 show that adaptive control can approach these best fixed-policy speeds while adding explicit rescue/fallback guardrails.

7. Discussion on Adaptive Speculative Strategies Derived from Empirical Findings

The current empirical study identifies the strongest fixed-policy speedup at smaller block size (k=4), with net speed degrading as k increases even when accepted block length rises. This indicates that fixed-policy drafting is vulnerable to acceptance/overhead imbalance and long-tail latency. This motivates a derived adaptive strategy guideline: Adaptive Cascaded Speculative Decoding (ACSD), which keeps the standard verifier but adapts policy online. In the latest implementation, ACSD is integrated into the Phase 7/8 pipeline with additional runtime-stability controls to reduce pathological latency tails in long runs.

ACSD is inspired by a practical deployment gap. Advanced speculative variants such as EAGLE-3 and DFlash reduce drafting cost through new drafter architectures and dedicated training/integration pipelines, and are typically reported on very strong accelerators. Our objective is different: retain a same-family drafter/verifier stack that can run on consumer hardware, then recover additional speed by adapting runtime policy (adaptive k, rescue switching, and tail fallback) instead of introducing a new trained drafter.

7.1. Design objective

Let speculative-cycle latency be

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \tag{4}$$

where τ is expected accepted tokens per cycle. In fixed-policy speculative decoding, poorly matched samples can reduce τ and increase total verify steps, producing heavy-tail runtime. ACSD explicitly targets this effect by adapting draft policy at sample level.

7.2. Proposed approach

ACSD combines four online controls while preserving standard target-side verification:

1. **Adaptive k:** choose $k \in \{3, 4, 5\}$ from rolling acceptance, using larger blocks only when acceptance supports it. To avoid cold-start overshoot, each sample starts from base k before adaptive updates.

2. **Cascaded draft switch:** use 0.5B as a fast primary drafter and switch to 1.5B rescue drafting when acceptance remains low for consecutive verify steps, then enforce a cooldown before a new rescue episode to reduce oscillation.

3. **Tail guardrail:** if acceptance stays below a threshold after a minimum generated length for multiple consecutive checks, fall back to target-only autoregressive decoding for the remainder of that sample.

4. **Checkpointed execution:** persist partial CSV and verify-log artifacts every fixed number of samples so interrupted long runs retain recoverable progress.

This design preserves verifier compatibility and allows direct comparison with vanilla speculative decoding under the same manifests, prompts, metrics, and hardware.

7.3. Evaluation plan

The extension is evaluated as a strict incremental study rather than a new protocol. We retain:

- the same target model and task manifests,

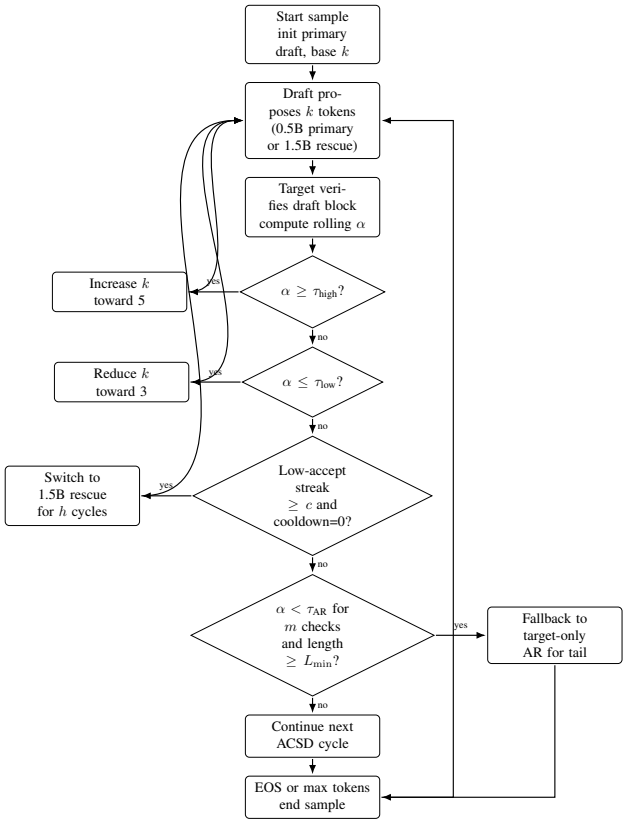

Figure 3. Adaptive Cascaded Speculative Decoding (ACSD) control loop. ACSD adapts block size from rolling acceptance, switches from the 0.5B primary to a 1.5B rescue drafter when acceptance remains low, applies rescue cooldown, and uses delayed target-only autoregressive fallback for pathological tails.

- the same deterministic and stochastic regimes,
- the same reporting metrics (S, α, B_{eff}, quality deltas),
- the same artifact schema for per-configuration CSV outputs, with ACSD metadata fields for adaptive-k usage, rescue usage, and fallback rate.

Primary comparison points are the current best fixed-policy speculative setting (k=4 deterministic) and the high-k settings (k=16) where fixed-policy decoding showed diminishing returns.

7.4. Empirical ACSD results

We ran ACSD on the full 1,000-sample manifests for both deterministic and stochastic regimes, using the same target model and evaluation protocol as the fixed-policy grid. Table 5 reports headline runtime metrics. Pairwise speedup S is recomputed from per-sample baseline and ACSD latencies matched by sample identifier and task.

The ACSD runtime is close to the best fixed-policy settings: 2.9% below best fixed deterministic speedup (3.0674) and 1.4% below best fixed stochastic speedup (2.9394). Rescue activation remains limited (0.698/0.843 mean rescue steps per sample) and tail fallback is rare (1.6%/2.2%), indicating that adaptive controls are used as guardrails rather than dominating the decode path.

Table 5. ACSD headline results on RTX4090 (0.5B primary, 1.5B rescue, adaptive $k \in \{3, 4, 5\}$). ΔS_{best} is ACSD speedup minus the best fixed-policy speedup in the same regime.

Regime	S	α	B_{eff}	R_{tok}	Rescue	Fallback	ΔS
deterministic	2.9792	0.4207	1.7977	32.61	0.698	1.6%	-0.08
stochastic	2.8981	0.4071	1.7214	31.08	0.843	2.2%	-0.04

Quality remains near baseline in deterministic ACSD (ΔGSM8K $= +0.33$, ΔMMLU $= 0.00$, ΔCNN/DM $= -0.02$), while stochastic ACSD shows moderate degradation (ΔGSM8K $= -1.00$, ΔMMLU $= -0.20$, ΔCNN/DM $= -0.22$). Aggregated adaptive-k traces show most cycles at $k \in \{4, 5\}$ (deterministic: 86.9%; stochastic: 85.8%), with k=3 used as a recovery mode and very short steps ($k \in \{1, 2\}$) mainly near completion.

8. Discussion

8.1. Main empirical takeaways

Three findings are consistent in the current run family. First, speculative decoding provides consistent acceleration across all tested configurations ($S > 1$ throughout). Second, speedup declines as k increases in this run family, with the strongest performance at k=4 and the weakest at k=16. Third, deterministic decoding outperforms stochastic decoding in 5 of 6 matched draft–k pairs.

Together, these results suggest that practical deployment should prioritise a smaller block size (k=4) and deterministic policy when latency is the primary objective.

The key engineering implication is that, on consumer-class hardware, speedup is governed by a balance between acceptance and per-cycle systems overhead. In our runs, B_{eff} grows with k while speedup still declines, indicating that memory-traffic and loop-overhead growth outpace useful acceptance gains at larger block sizes.

The ACSD extension adds a second practical option: adaptive control keeps speed close to best fixed-policy settings while providing explicit rescue and tail-fallback mechanisms. On the current run family, ACSD reaches S=2.9792 (deterministic) and S=2.8981 (stochastic), with low fallback activation (1.6% and 2.2%).

From a positioning standpoint, this matters because demand for local and consumer-hardware LLM inference is increasing: the relevant question is no longer only "how fast can we decode in data centers," but "which decoding policy remains efficient and stable on constrained personal hardware."

For this reason, our recommendations are framed as

hardware-profiled operating points rather than universal defaults.

8.2. Draft-size implications

The 0.5B draft produces the best single configuration (0.5B, k=4, deterministic), and also has a slightly higher mean speedup than 1.5B across the full grid in this run family. This indicates that draft-size selection is hardware- and budget-sensitive: the smaller draft can win on end-to-end speed even when the larger draft improves acceptance.

8.3. Speed versus quality

Quality drift is task-dependent. CNN/DailyMail and MMLU remain relatively stable across settings, while GSM8K is more variable, especially under stochastic decoding. A conservative filter ($\max|\Delta| \leq 1$ across all three tasks) retains a small subset of configurations, among which 1.5B, k=8, deterministic remains best. This supports a practical policy: use deterministic mode with k=4 for production-like latency targets, and treat stochastic settings as quality-sensitive configurations requiring additional calibration. Final winner claims should be interpreted together with paired confidence intervals and corrected pairwise tests from a coherent artifact family.

9. Threats to validity

Artifact consistency across reruns. Some baseline files were regenerated after initial summary artifacts were produced, which can create mismatches between raw baseline totals and summary-implied baselines. To mitigate this, we use one coherent summary source for comparative claims and report this limitation explicitly.

Single-hardware scope. All results are from one RTX 4090 Laptop. Relative rankings are informative for this deployment class, but absolute speedups may shift on other GPU generations. In this run family, CUDA-graph metadata indicates pending/zero capture rate, so reported gains reflect the same code path under fallback execution rather than successful graph replay.

Model-family scope. All pairings are same-family (Qwen2.5 target and drafts). Cross-family pairings may produce materially different acceptance and quality behaviour.

Evaluation breadth. The benchmark set spans math reasoning, knowledge QA, and summarisation, but does not exhaust safety, factuality, or long-context behaviour. ROUGE-L in particular is an incomplete summarisation quality proxy.

10. Conclusion

This paper provides a controlled consumer-GPU evaluation of speculative decoding with a Qwen2.5-3B target and same-family drafts. The current evidence shows consistent acceleration across all tested configurations, with best observed performance at 0.5B, k=4, deterministic (S=3.0674). We find that increasing k from 4 to 8 and 16 reduces net speedup in this setup, and that deterministic decoding is generally faster than stochastic decoding for matched settings.

These findings yield an actionable operating recommendation for the present hardware budget: prefer deterministic speculative decoding at k=4, with 0.5B draft as the default fast path and larger drafts reserved for quality-sensitive scenarios. The ACSD results (Section 7) show that adaptive draft length and cascaded draft-model switching can deliver near-best fixed-policy speed while adding low-frequency rescue/fallback controls for tail robustness. This supports ACSD as a practical consumer-GPU policy layer rather than a replacement for the fixed-policy fast path.

References

[1] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum. PARD: Accelerating llm inference with low-cost parallel draft model adaptation. *arXiv preprint arXiv:2504.18583*, 2025. 2

[2] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. 2

[3] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. In *arXiv preprint arXiv:2302.01318*, 2023. 1, 2

[4] J. Chen, Y. Liang, and Z. Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026. 2

[5] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 3

[6] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. 3

[7] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. 1, 2

[8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang. DiffuSpec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358v1*, 2025. https://arxiv.org/abs/2510.02358v1. 2

[9] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024. 2

[10] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024. 2

[11] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025. 2

[12] S. Narayan, S. B. Cohen, and M. Lapata. Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. 3

[13] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto. SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding. *arXiv preprint arXiv:2511.00606*, 2025. 2

[14] M. Stern, N. Shazeer, and J. Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2